

# INFORME DEL TRABAJO PRÁCTICO

Trabajo Práctico: Super Elizabeth Sis

UNIVERSIDAD NACIONAL DE  
GENERAL SARMIENTO

Materia: Programación I (2026)

**Grupo:** 11

**Integrantes:**

- Ignacio Pianezz; 44724357;
- Lautaro Lang; 47.386.120; [lautarolang34@gmail.com](mailto:lautarolang34@gmail.com)
- Lucas Albornoz; 45.738.132;

## Introducción

En este trabajo práctico les presentamos "Super Elizabeth Sis", un juego de plataformas en 2D inspirado en el Mario Bros. Manejamos a la princesa Elizabeth, que tiene que ir saltando por plataformas, esquivando o matando murciélagos a los tiros, hasta llegar sana y salva al castillo para ganar. Todo esto lo armamos en Java usando las clases Entorno y Herramientas que nos dio la cátedra para manejar la pantalla, los gráficos y el teclado.

## Descripción de las Clases

Dividimos el código en varias clases para organizar bien los objetos:

**Juego:** Es la clase principal. Acá adentro tenemos el método `tick()` que se ejecuta todo el tiempo. En esta clase manejamos los arreglos de las islas y los enemigos, detectamos los choques y controlamos si ganás o perdés.

**Princesa:** Es nuestro personaje principal. Tiene variables para guardar su posición (x e y), su tamaño, la velocidad de salto y cuántas vidas le quedan. Además tiene los métodos para moverse y ver si choca contra algo.

**Isla:** Son las plataformas. Las armamos de forma intercalada: unas actúan de piso y otras flotan en el cielo.

**Enemigo:** Son los murciélagos. Los manejamos con un arreglo. Le sumamos un detalle extra para subir nota (un requerimiento opcional): tienen un 20% de probabilidad de nacer como "Enemigos de Élite". Se pintan de azul, vuelan más rápido y te sacan 2 vidas en vez de 1.

**Castillo:** Es la meta del nivel. Si la princesa choca con esto, cortamos el juego y mostramos la pantalla de victoria.

**Proyectil:** Son los disparos de Elizabeth. Cuando salen de la pantalla o le pegan a un murciélago, desaparecen (se vuelven `null`).

**Problemas que tuvimos y cómo los arreglamos:** Durante el desarrollo renegamos con un par de cosas, pero las pudimos sacar adelante:

**Murciélagos adentro de la tierra:** Al principio, como nacían en lugares al azar, a veces aparecían fusionados con las plataformas. Lo solucionamos armando un "Spawn Check": creamos al enemigo de forma temporal y nos fijamos si choca con una isla al nacer; si choca, lo descartamos y el ciclo vuelve a intentar crear otro en el próximo tick.

**El castillo quedaba flotando:** Queríamos que el castillo se armara justo arriba de la última isla, pero a veces la última isla del arreglo caía impar (flotante) y el castillo nos quedaba en las nubes. Lo solucionamos mirando el tamaño del

arreglo (`islas.length - 1`): si la última era flotante, le decíamos al código que retroceda un índice y tomara las coordenadas de la última isla que sí era piso firme.

**Pantalla negra (Crash):** Quisimos meter un sprite de Goku y de la nada el juego quedaba todo negro y no abría. Mirando la consola descubrimos que nos tiraba un `NullPointerException` porque Java no encontraba el archivo. Resulta que le habíamos puesto un `.png` en la carpeta equivocada. Lo convertimos a `.gif`, lo dejamos suelto en la carpeta raíz del proyecto, le dimos a "Refresh" en Eclipse y arrancó de diez.

## Implementación

Clase Juego;

```
public class Juego extends InterfaceJuego
{
    private Entorno entorno;

    private Princesa elizabeth;

    private Isla[] islas;

    private Isla ultimaIslaSegura;

    private Enemigo[] enemigos;

    private Disparo ataque;

    private Castillo castillo;

    Juego()
    {
        this.entorno = new Entorno(this, "Proyecto para TP", 800, 600);

        this.entorno.iniciar();
        this.elizabeth = new Princesa(400, 400);
        this.enemigos = new Enemigo [7];
        this.islas = new Isla[30];

        int posX = 400;
        for (int i = 0; i < islas.length; i++) {

            if (i % 2 == 0) {
                this.islas[i] = new Isla(posX, 550, 200, 30);
                posX = posX + 280;
                this.castillo = new Castillo(posX, 485);
            } else {
                int alturaAleatoria;
                if (Math.random() > 0.5) {
                    alturaAleatoria = 400;
                } else {
                    alturaAleatoria = 250;
                }
                this.islas[i] = new Isla(posX - 140, alturaAleatoria, 200, 20);}
        }
    }
}
```

```

    }
    Isla ultimaIsla = islas[islas.length - 1];
    double posYCastillo = ultimaIsla.y - ultimaIsla.alto / 2 - 50;
    this.castillo = new Castillo(ultimaIsla.x, posYCastillo); }
    public void tick()
    {

        if (elizabeth.vidas < 1) {
            entorno.cambiarFont("Arial", 60, Color.RED);

            entorno.escribirTexto("GAME OVER", 200, 300);

            return;

        }
        if (castillo != null && elizabeth.chocaConCastillo(castillo)) {
            entorno.cambiarFont("Arial", 60, Color.GREEN);

            entorno.escribirTexto("¡GANASTE!", 250, 300);

            return;
        }
        entorno.cambiarFont("Arial", 24, Color.RED);
        entorno.escribirTexto("Vidas: " + elizabeth.vidas, 50, 50);
        elizabeth.dibujar(entorno);
        for (int i = 0; i < islas.length; i++) {
            if (islas[i] != null) {
                islas[i].dibujar(entorno);
            }
        }
    }

    if (entorno.estaPresionada(entorno.TECLA_DERECHA)) {
        if (elizabeth.x < 400) {
            elizabeth.moverDerecha();
        } else {

            for (int i = 0; i < islas.length; i++) {
                if (islas[i] != null) {
                    islas[i].x = islas[i].x - 3;
                }
            }

            for (int i = 0; i < enemigos.length; i++) {
                if (enemigos[i] != null) {
                    enemigos[i].x = enemigos[i].x - 3;
                }
            }
            if (castillo != null) {
                castillo.x = castillo.x - 3;
            }
        }
    }
}

if (entorno.estaPresionada(entorno.TECLA_IZQUIERDA)) {
    if (elizabeth.x > 15) {
        elizabeth.moverIzquierda();
    }
}
}

```

```

double yAnterior = elizabeth.y;

boolean estaApoyada = false;
Isla islaDondeSeApoya = null;

for (int i = 0; i < islas.length; i++) {
    if (islas[i] != null && elizabeth.estaApoyadaSobre(islas[i])) {
        estaApoyada = true;
        islaDondeSeApoya = islas[i];
    }
}

if (entorno.estaPresionada(entorno.TECLA_ARRIBA) && estaApoyada) {
    elizabeth.salto();
}

elizabeth.actualizarsalto();

boolean estaSubiendo = elizabeth.y < yAnterior;

if (!estaApoyada && !estaSubiendo) {
    elizabeth.estaenelAire = true;
    elizabeth.caer();
}
if (estaSubiendo) {
    for (int i = 0; i < islas.length; i++) {

        if (islas[i] != null && elizabeth.chocaCabezaCon(islas[i])) {

            elizabeth.velocidad = 0;

            elizabeth.y = islas[i].y + islas[i].alto / 2 + elizabeth.alto / 2;
        }
    }
}

if (estaApoyada && !estaSubiendo) {
    elizabeth.apoyarSobre(islaDondeSeApoya);
    elizabeth.estaenelAire = false;
    ultimaIslaSegura = islaDondeSeApoya;
}

if (elizabeth.y > entorno.alto()) {

    elizabeth.x = ultimaIslaSegura.x;
    elizabeth.y = 100;
    elizabeth.vidas = elizabeth.vidas - 1;
}
for (int i = 0; i < enemigos.length; i++) {
    if (enemigos[i] == null) {
        if (Math.random() < 0.01) {

```

```

        double alturaAleatoria;
        do {
            alturaAleatoria = Math.random() * 500 + 50;
        } while (
            alturaAleatoria > 220 && alturaAleatoria < 280 ||
            alturaAleatoria > 370 && alturaAleatoria < 430 ||
            alturaAleatoria > 520 && alturaAleatoria < 580
        );

        double posX;
        if (Math.random() > 0.5) {
            posX = 800;
        } else {
            posX = 0;
        }

        enemigos[i] = new Enemigo(posX, alturaAleatoria);
    }
} else {
    enemigos[i].dibujar(entorno);
    enemigos[i].mover();
    if (elizabeth.chocaCon(enemigos[i])) {
        elizabeth.vidas = elizabeth.vidas - enemigos[i].danio;
        enemigos[i] = null;
    }
    else if (enemigos[i].x < 0 || enemigos[i].x > 800) {
        enemigos[i] = null;
    }
    if (entorno.sePresionoBoton(entorno.BOTON_IZQUIERDO) && ataque == null) {
        double angulo = Math.atan2(entorno.mouseY() - elizabeth.y, entorno.mouseX() -
elizabeth.x);
        ataque = new Disparo(elizabeth.x, elizabeth.y, angulo);
    }
    if (ataque != null) {
        ataque.dibujar(entorno);
        ataque.mover();
        if (ataque.x < 0 || ataque.x > entorno.anch() || ataque.y < 0 || ataque.y >
entorno.alto()) {
            ataque = null;
        }
        else {
            for (int j = 0; j < enemigos.length; j++) {
                if (enemigos[j] != null && ataque != null && ataque.chocaCon(enemigos[j])) {
                    enemigos[j] = null;
                    ataque = null;
                }
            }
        }
    }
    if (castillo != null) {
        castillo.dibujar(entorno);
    }
}
}
}
@SuppressWarnings("unused")

```

```

    public static void main(String[] args)
    {
        Juego juego = new Juego();
    }
}

```

## Clase Princesa;

```

public class Princesa {

    double x;
    double y;
    double ancho;
    double alto;
    double velocidad;
    boolean estaenelAire;
    int vidas;
    //constructor
    public Princesa(double xInicial, double yInicial) {
        this.x = xInicial;
        this.y = yInicial;
        this.ancho = 30;
        this.alto = 50;
        this.velocidad = 0;
        this.estaenelAire = false;
        this.vidas = 3;
    }
    // metodo para dibujarse con el entorno
    public void dibujar(Entorno entorno) {
        entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, Color.PINK);
    }
    // movilidad
    public void moverDerecha() {
        this.x = this.x + 3;
    }
    public void moverIzquierda() {
        this.x = this.x - 3;
    }
    public void caer() {
        this.y = this.y + 5; } //caida
    public boolean estaApoyadaSobre(Isla isla) {
        boolean coincideX = this.x + this.ancho / 2 > isla.x - isla.ancho / 2 &&
            this.x - this.ancho / 2 < isla.x + isla.ancho / 2;

        boolean coincideY = this.y + this.alto / 2 >= isla.y - isla.alto / 2 &&
            this.y + this.alto / 2 <= isla.y + isla.alto / 2;
        return coincideX && coincideY;
    }
    public void apoyarSobre(Isla isla) {
        y = isla.y - isla.alto / 2 - alto / 2;
        velocidad = 0;
    }

    public void salto() {
        velocidad = -12;
    }
}

```

```

        estaenelAire= true;
    }

    public void actualizarSalto() {
        y = y + velocidad;
        velocidad = velocidad + 0.5;
    }

    public boolean chocaCabezaCon(Isla isla) {
        boolean coincideX = this.x + this.anchos / 2 > isla.x - isla.anchos / 2 &&
            this.x - this.anchos / 2 < isla.x + isla.anchos / 2;

        boolean coincideY = this.y - this.alto / 2 <= isla.y + isla.alto / 2 &&
            this.y - this.alto / 2 >= isla.y - isla.alto / 2;

        return coincideX && coincideY;
    }

    public boolean chocaCon(Enemigo enemigo) {
        boolean colisionX = this.x - this.anchos / 2 < enemigo.x + enemigo.anchos / 2 &&
            this.x + this.anchos / 2 > enemigo.x - enemigo.anchos / 2;

        boolean colisionY = this.y - this.alto / 2 < enemigo.y + enemigo.alto / 2 &&
            this.y + this.alto / 2 > enemigo.y - enemigo.alto / 2;
        return colisionX && colisionY;
    }

    public boolean chocaConCastillo(Castillo castillo) {
        boolean colisionX = this.x - this.anchos / 2 < castillo.x + castillo.anchos / 2 &&
            this.x + this.anchos / 2 > castillo.x - castillo.anchos / 2;
        boolean colisionY = this.y - this.alto / 2 < castillo.y + castillo.alto / 2 &&
            this.y + this.alto / 2 > castillo.y - castillo.alto / 2;
        return colisionX && colisionY;
    }
}

```

## Clase Isla;

```

public class Isla {
    double x;
    double y;
    double anchos;
    double alto;
    public Isla(double x, double y, double anchos, double alto) {
        this.x = x;
        this.y = y;
        this.anchos = anchos;
        this.alto = alto;
    }
    public void dibujar(Entorno entorno) {
        entorno.dibujarRectangulo(this.x, this.y, this.anchos, this.alto, 0, Color.GREEN);
    }
}

```

### Clase Enemigo;

```
public class Enemigo {
    double x;
    double y;
    double ancho;
    double alto;
    double velocidad;
    int danio;
    Color color;
    public Enemigo(double x, double y) {
        this.x = x;
        this.y = y;
        this.ancho = 30;
        this.alto = 30;
        this.danio = 1;
        this.color = Color.RED;

        if (this.x < 400) {
            this.velocidad = 2;
        }
        else {
            this.velocidad = -2;
        }
        if (Math.random() < 0.20) {
            this.danio = 2;
            this.velocidad = this.velocidad * 1.5;
            this.color = Color.BLUE;
        }
    }
    public void dibujar(Entorno entorno) {
        entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, this.color);
    }
    public void mover() {
        this.x = this.x + this.velocidad;
    }
}
```

### Clase Disparo;

```
public class Disparo {
    double x;
    double y;
    double angulo;
    double velocidad;
    double radio;
    public Disparo(double x, double y, double angulo) {
        this.x = x;
        this.y = y;
        this.angulo = angulo;
        this.velocidad = 3;
        this.radio = 15;
    }
    public void dibujar(Entorno entorno) {
        entorno.dibujarCirculo(this.x, this.y, this.radio * 2, Color.ORANGE);
    }
    public void mover() {
```

```

    this.x = this.x + Math.cos(this.angulo) * this.velocidad;
    this.y = this.y + Math.sin(this.angulo) * this.velocidad;

}
public boolean chocaCon(Enemigo enemigo) {
    boolean colisionX = this.x - this.radio < enemigo.x + enemigo.ancho / 2 &&
        this.x + this.radio > enemigo.x - enemigo.ancho / 2;

    boolean colisionY = this.y - this.radio < enemigo.y + enemigo.alto / 2 &&
        this.y + this.radio > enemigo.y - enemigo.alto / 2;
    return colisionX && colisionY;
}
}

```

Clase Castillo;

```

public class Castillo {
    double x;
    double y;
    double ancho;
    double alto;
    public Castillo(double x, double y) {
        this.x = x;
        this.y = y;
        this.ancho = 100;
        this.alto = 100;
    }
    public void dibujar(Entorno entorno) {
        entorno.dibujarRectangulo(this.x, this.y, this.ancho, this.alto, 0, Color.YELLOW);
    }
}

```

## Conclusiones

La verdad que armar el juego fue todo un desafío pero estuvo muy bueno. Al principio costó cazarle la onda a cómo se ejecutaban las cosas adentro del ciclo `tick()`, y tuvimos que estar súper atentos a los arreglos para evitar que el programa se rompa por intentar leer posiciones vacías (el famoso `NullPointerException`). También nos sirvió un montón para practicar con Git y trabajar en equipo. Creemos que logramos un juego divertido, funcional y que cumple con todos los pedidos de la cátedra.